

Lab 4 — Submission Setup, and Object Oriented Programming

git, from scratch, and your first push — then the OOP exercises

Amir Farbin

Today

Two halves

- ▶ **Part 1 — Submission setup.** Fork, clone, wire the remotes, push. We do this together, from scratch, right now. **Lab 2 is due tonight, and you submit it here.**
- ▶ **Part 2 — Object Oriented Programming.** The lab itself: 12 exercises, using the Lectures 8–9 material.

Two halves

- ▶ **Part 1 — Submission setup.** Fork, clone, wire the remotes, push. We do this together, from scratch, right now. **Lab 2 is due tonight, and you submit it here.**
- ▶ **Part 2 — Object Oriented Programming.** The lab itself: 12 exercises, using the Lectures 8–9 material.
- ▶ Labs/GitHub-Setup.pdf — the same steps as Part 1, written out line by line, with a table of what to do when git complains
- ▶ Labs/Lab.4/Lab.4.ipynb — everything below, plus the exercises, in one notebook

git — what it is, and the words you'll use

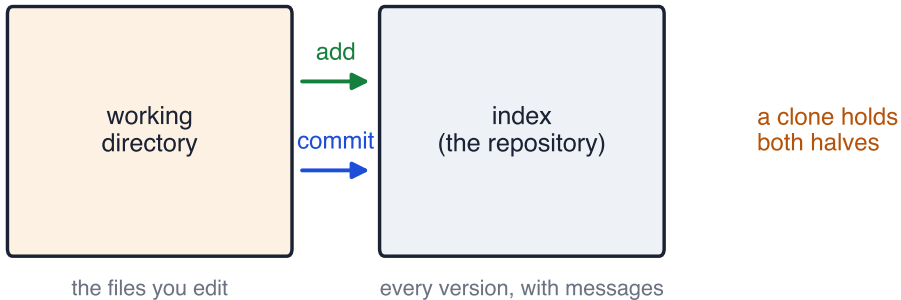
What git is

- ▶ A **version control system** — it tracks how files change over time
- ▶ Built for code, good for any large document
- ▶ Lets several people work on one thing without overwriting each other
- ▶ Lets you keep parallel versions, and mark releases
- ▶ Works entirely on your own machine, or against a server

What git is

- ▶ A **version control system** — it tracks how files change over time
- ▶ Built for code, good for any large document
- ▶ Lets several people work on one thing without overwriting each other
- ▶ Lets you keep parallel versions, and mark releases
- ▶ Works entirely on your own machine, or against a server
- ▶ **GitHub** runs a public git server, and has become how open-source code is shared
- ▶ **GitLab** is a similar thing companies often host themselves

A repository holds two things



- ▶ **add** puts a file's changes in line to be recorded
- ▶ **commit** records them, with a message saying what you did

The words you'll see

- ▶ **Clone** — your own local copy of a repository. What you actually work in
- ▶ **Remote** — a copy living on a server, that your clone talks to
- ▶ **Fork** (*a GitHub feature, not a git command*) — a copy of someone else's repository on *your* GitHub account, that can now go its own way
- ▶ **Branch** — a parallel version that doesn't disturb the main one (`main`)
- ▶ **Tag** — a name pinned to one particular version

The words you'll see

- ▶ **Clone** — your own local copy of a repository. What you actually work in
- ▶ **Remote** — a copy living on a server, that your clone talks to
- ▶ **Fork** (*a GitHub feature, not a git command*) — a copy of someone else's repository on *your* GitHub account, that can now go its own way
- ▶ **Branch** — a parallel version that doesn't disturb the main one (`main`)
- ▶ **Tag** — a name pinned to one particular version

- ▶ **fetch** — get changes from a remote · **merge** — fold them into your files
- ▶ **pull** = fetch + merge · **push** — send your commits to a remote
- ▶ **Merge conflict** — when two people changed the same thing and git can't decide. Whoever pushes second resolves it (*there's a trick to avoid this in this course entirely — next section*)
- ▶ **Pull request** — asking to pull *your* fork's changes back into someone else's repo. Not something you'll do in this course — we only push to our own fork — but you'll see the term everywhere git is used for real

Part 1 — Submission Setup

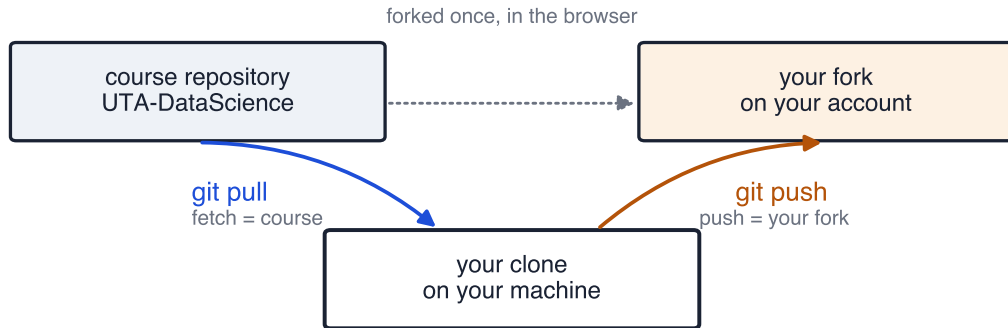
Why a fork, specifically

- ▶ The course repository is **public** — `UTA-DataScience/DATA3402.Fall.2026`
- ▶ You don't have write access to it, so you need your own copy to push to — that's what the fork is for

Why a fork, specifically

- ▶ The course repository is **public** — UTA-DataScience/DATA3402.Fall.2026
- ▶ You don't have write access to it, so you need your own copy to push to — that's what the fork is for
- ▶ Because the class repo is public, **your fork will be public too**. That's expected.

How this course is wired



one remote, named `origin` — it fetches from one place and pushes to another

Pull from class. Push to my fork. Never force push.

Setup — once per semester

1. Fork the class repo, in your browser
2. `git clone` your fork onto your machine
3. `cd` into the clone
4. `git remote remove origin`
5. `git remote add origin <class repo URL>`
6. `git remote set-url --push origin <your fork's URL>`

Setup — once per semester

1. Fork the class repo, in your browser
2. `git clone` your fork onto your machine
3. `cd` into the clone
4. `git remote remove origin`
5. `git remote add origin <class repo URL>`
6. `git remote set-url --push origin <your fork's URL>`

Check it: `git remote -v` should show the **class repo for fetch**, and **your fork for push**. If both lines show the same URL, step 6 didn't take — run it again.

Every week

every lab, the same five steps



Avoid the merge conflict before it happens

**The moment you `git pull` a new lab, copy it to a new name and work there
— never edit the file we handed you.**

```
cp Lab.2.ipynb Lab.2.solution.ipynb
```

Avoid the merge conflict before it happens

The moment you `git pull` a new lab, copy it to a new name and work there — never edit the file we handed you.

`cp Lab.2.ipynb Lab.2.solution.ipynb`

- ▶ `git pull` only touches files it recognizes from the class repo. `Lab.2.ipynb` is one of those; `Lab.2.solution.ipynb` isn't — we never handed out a file by that name
- ▶ So a later `git pull`, even one that updates `Lab.2.ipynb` after we fix something in it, **can never conflict with your copy**. There's nothing shared left to disagree about
- ▶ Do this **first**, before you write a line of code — not at submission time. If you've already been editing `Lab.2.ipynb` directly, copy it now and switch to the copy

Submitting

1. `git pull` — get this week's material
2. Confirm you've been working in `Lab.2.solution.ipynb`, not `Lab.2.ipynb` — if not, copy it over now, before you do anything else
3. `git add Labs/Lab.2/Lab.2.solution.ipynb`
4. `git commit -a -m "Lab 2 submission"`
5. `git push`
6. Check GitHub — your fork should show the new commit

If it goes wrong

- ▶ **destination path already exists** on the clone — you still have the Lab 1 clone of the class repo. Don't clone again or delete it: `cd` into it and carry on from step 4
- ▶ **rejected (non-fast-forward)** on your very first push — expected. Your fork and the class repo have separate histories: `git config pull.rebase false`, pull, then push again
- ▶ **Merge conflict** on `git pull` — you edited a file we handed out, and we changed it too. Nothing is lost. Don't hand-edit the JSON — `Labs/GitHub-Setup.pdf` has the steps that keep your work under its own name and get you pulling again
- ▶ Full error table, line by line: `Labs/GitHub-Setup.pdf`

Before you leave today

- ▶ Confirm your fork shows the pushed commit on GitHub
- ▶ Need to keep working later? Edit the same clone, then repeat add/commit/push

Part 2 — Object Oriented Programming

- ▶ **Encapsulate** — bundle data and the operations on it into one object
- ▶ **Public / private** — by convention, a leading `__` means “internal to the class”; everything else goes through accessor functions you write
- ▶ **Inheritance** — a base class states the common interface; subclasses fill it in
- ▶ **Canvas, Shape, RasterDrawing** — the drawing classes from Lecture 9 are exactly what exercises 10–12 build on

The arc of the 12 exercises

- ▶ **1–2** — a counter class, then the same thing with private data
- ▶ **3–6** — rectangle, circle, a shared base class, triangle
- ▶ **7–9** — perimeter points, point-containment, overlap
- ▶ **10–12** — paint it: a Canvas/paint module, RasterDrawing, save/load

The arc of the 12 exercises

- ▶ **1–2** — a counter class, then the same thing with private data
- ▶ **3–6** — rectangle, circle, a shared base class, triangle
- ▶ **7–9** — perimeter points, point-containment, overlap
- ▶ **10–12** — paint it: a Canvas/paint module, RasterDrawing, save/load

Each exercise builds on the last. A shaky base class in 5 doesn't just break 5 — it makes 6 through 9 harder too. Get help early rather than push on with it.

Save and load, the trick

- ▶ Overload `__repr__` to return the **Python code that would reconstruct the object**
- ▶ Saving is writing that string to a file
- ▶ Loading is reading the string back and eval-ing it

Save and load, the trick

- ▶ Overload `__repr__` to return the **Python code that would reconstruct the object**
- ▶ Saving is writing that string to a file
- ▶ Loading is reading the string back and eval-ing it

Labs/Lab.4/Lab.4.ipynb has a worked `foo / foo_loader` example at the very end — read it before you start exercise 12, not after.